

Relatório de Integração — Sprint 2

Ferramenta escolhida: RabbitMQ.

Escolhido por ser o MOM mais utilizado em ambientes corporativos, ter suporte nativo ao protocolo AMQP, oferecer painel de administração visual e é facilmente executado via Docker sem configuração adicional.

Padrão utilizado: Produtor/Consumidor com filas diretas (Direct Queue). Cada tipo de evento possui sua própria fila nomeada. O backend atua como produtor e um processo separado atua como consumidor.

Dois eventos implementados:

`incident.created` — publicado quando um novo incidente é criado

`incident.status_updated` — publicado quando o analista atualiza o status

Por que dois processos separados: O consumidor roda em processo independente do Flask, demonstrando que a comunicação é genuinamente assíncrona. O backend não chama o consumidor diretamente, apenas publica na fila.

Desafios encontrados: Garantir que o backend não quebrasse caso o RabbitMQ estivesse fora do ar. Resolvido com `try/except` no método `publish()`, isolando a falha do MOM do fluxo principal da API.